JWT & MYSQL

```
const express = require("express");
const jwt = require("jsonwebtoken");
const dotenv = require("dotenv");

dotenv.config();

const app = express();

app.use(express.json());

const user = {
  id: 1,
  username: "ravi",
  password: "12345",
};

app.post("/login", (req, res) => {
  const { username, password } = req.body;

  if (username === user.username && password === user.password) {
    const token = jwt.sign(
      { id: user.id, username: user.username },
      process.env.SECRET,
      { expiresIn: "1h" },
    );
    res.json({ token });
  } else {
    res.send("invalid user name and password");
  }
});

const authMiddleware = async (req,res,next )=>{
  const token = req.headers["authorization"]
  console.log(req.headers);
  console.log(req.headers.authorization);

  if (!token) return res.json({ message: "Access denied" });
  try {
    const verified = jwt.verify(token, process.env.SECRET);
    req.user = verified;
    next();
  } catch (err) {
    res.status(400).json({ message: "Invalid token" });
  }
};
```

```
app.get("/profile", authMiddleware, (req, res) => {
  res.json({ message: "Welcome to your file", user: req.user });
});

app.listen(process.env.PORT, () => {
  console.log("Server is up and running");
});
```

sql

```
--  dcl data control language

-- ravi and raj

-- ravi => owner , table, delete,update,  drop , select
-- raj => staff  select

use raviraj;

GRANT SELECT ON user TO 'raj'@'localhost';

REVOKE SELECT ON user FROM 'raj'@'localhost';

-- TCL --> transaction control language
START transaction;
delete from user WHERE id =6;
commit;
rollback;



CREATE TABLE users (
  id INT PRIMARY KEY AUTO_INCREMENT,
  name VARCHAR(50) NOT NULL,
  email VARCHAR(100) UNIQUE,
  age INT
);

CREATE TABLE orders (
  id INT PRIMARY KEY AUTO_INCREMENT,
  user_id INT,
  product VARCHAR(50),
  amount INT,
  FOREIGN KEY (user_id) REFERENCES users(id)
);
```

```sql
INSERT INTO users (name, email, age) VALUES
('Ejaz', 'ejaz@example.com', 25),
('Sara', 'sara@example.com', 30),
('Ali', 'ali@example.com', 28);

INSERT INTO users (name, email, age) VALUES
('Raviraj', 'ravi@example.com', 25);

INSERT INTO orders (user_id, product, amount) VALUES
(1, 'Laptop', 800),
(1, 'Mouse', 20),
(2, 'Phone', 500),
(3, 'Tablet', 300);


INSERT INTO orders (user_id, product, amount) VALUES
(5, 'Laptop', 1000);


SELECT COUNT(*) as total_user FROM users;
SELECT AVG(amount) as avg_order_amount  FROM orders;
SELECT MAX(amount) as highest_order FROM orders;
SELECT MIN(amount) as lowest_amount FROM orders;


SELECT users.name, orders.product, orders.amount
FROM users
JOIN orders ON users.id = orders.user_id;


SELECT name FROM users WHERE id IN(
SELECT user_id FROM orders where product ="Laptop"
);
```

Theory Notes

1. SQL DCL (Data Control Language)

- **GRANT** → gives privileges to a user (e.g., `GRANT SELECT ON user TO 'raj'@'localhost';`).
- **REVOKE** → removes privileges (e.g., `REVOKE SELECT ON user FROM 'raj'@'localhost';`).
- Used for **security and access control** in databases.

2. SQL TCL (Transaction Control Language)

- **START TRANSACTION** → begins a transaction.

- **COMMIT** → saves changes permanently.
- **ROLLBACK** → undoes changes since the last commit.
- Ensures **data integrity** when multiple operations are performed together.

3. SQL Joins

- **INNER JOIN** → returns rows with matching values in both tables.
- Example:

sql
```
SELECT users.name, orders.product, orders.amount
FROM users
JOIN orders ON users.id = orders.user_id;
```

- Useful for combining data across related tables.

4. SQL Aggregate Functions

- **COUNT()** → number of rows.
- **AVG()** → average value.
- **MAX()** → highest value.
- **MIN()** → lowest value.

5. Express.js + JWT Authentication

- **JWT (JSON Web Token)** → stateless authentication mechanism.
- `jwt.sign(payload, secret, options)` → creates token.
- `jwt.verify(token, secret)` → validates token.
- Middleware (`authMiddleware`) checks token before allowing access to protected routes.
- `.env` file stores secrets (`SECRET`, `PORT`) securely.

Practice Tasks

SQL Coding Exercises

1. Create a new table `products` with columns `id`, `name`, `price`.
2. Insert 3 products and write a query to find the product with the highest price.
3. Write a query to count how many orders each user has placed (`GROUP BY`).
4. Use a subquery to find all users who ordered a "Laptop".
5. Grant `INSERT` privilege on `orders` table to user `raj`.

Express.js Coding Exercises

1. Add a `/register` route that stores new users in MySQL with hashed passwords.
2. Modify `login` to check credentials against the database instead of hardcoded values.

3. Update `authMiddleware` to expect `Bearer <token>` format in the `Authorization` header.
4. Add a `/logout` route that invalidates tokens (e.g., by maintaining a blacklist).
5. Create a role-based middleware that only allows `"admin"` users to access `/profile`.